

# CIS 301: Logical Foundations of Programming

Spring 2026

Exam 3 – 100 points

**This test is closed-notes and closed-computers.**

This exam has 10 multiple choice and 4 open-ended problems.

Name: \_\_\_\_\_ **Solution** \_\_\_\_\_

Score: \_\_\_\_\_

**Multiple choice – 10 questions, 3 points each.**

**For each question, circle the ONE option that is the best answer.**

1. What do we assume at the beginning of an ExistsE subproof?

- a) That whatever statement we are trying to prove is true for some random individual
- b) That a previous there-exists statement is true for some anonymous individual**
- c) That a previous there-exists statement is true for some known individual
- d) We do not start an ExistsE subproof with an assume statement

2. If we wish to prove that the sum of two even integers is even using proof by contradiction, how should we start?

- a) Let x and y be even integers. Assume, for the sake of contradiction, that x+y is odd.**
- b) Assume, for the sake of contradiction, that x and y are odd integers.
- c) Assume x+y is odd for integers x and y.
- d) Assume, for the sake of contradiction, that x+y is odd for integers x and y.

3. Consider the proof block below, where the justifications for line 1 is missing. Which of the following possible claims would be valid on line 2 in that proof block, with the justification of: **Algebra\*(1)**.

```
Deduce(  
  1  $\neg(a > 4)$  by SomeJustification,  
  2 ( ??? ) by Algebra*(1)  
)
```

- a)  $a < 0$
- b)  $a < 4$
- c)  $a < 10$**
- d) None of the above

4. Consider the loop below (assume *bound* is a previously defined variable). What is the most fitting invariant that describes the progress of the loop?

```
var x: Z = 0
var y: Z = bound
while (x < bound) {
    Invariant(
        Modifies(x, y),
        ???
    )

    y = y - 5
    x = x + 1
}
```

- a)  $y == y - 5$
- b)  $y == 5 * x$
- c)  $x < \text{bound}$
- d)  **$y == \text{bound} - 5 * x$**

5. Where do we need to PROVE that a loop invariant is true?

- a) After calling a function that contains a loop
- b) At the beginning of each iteration
- c) **Before the loop begins**
- d) After the loop has finished

6. Suppose we wish to use proof by contrapositive to prove that if  $x^2$  is odd for some integer  $x$ , then  $x$  is also odd. How should we start?

- a) Suppose, for the sake of contradiction, that  $x^2$  is even for some integer  $x$ .
- b) Suppose  $x^2$  is odd for some integer  $x$ . Suppose, for the sake of contradiction, that  $x$  is even.
- c) **Let  $x$  be an even integer.**
- d) Let  $x$  be an odd integer.

In Problems 7-10, consider the following program:

```
def example(c:Z, d:Z): Z = {  
  Contract(  
    Requires( c-3 > 12  $\wedge$  d < c*2 ),  
    Ensures( Res[Z] == 3*c + 2*(d-1) )  
  )  
  ...(code/verification for function)  
  
  return answer  
}  
////////// Test code //////////  
  
var m: Z = 12  
var n: Z = 3  
var ans: Z = example(m, m+n)
```

7. What must we prove before we call *example* in the test code?
  - a)  $c-3 > 12 \wedge d < c*2$
  - b)  $m-3 > 12 \wedge n < m*2$
  - c)  $9 > 12 \wedge 15 < 24$
  - d)  $m-3 > 12 \wedge m+n < m*2$**
  
8. What can we claim as a premise just inside the *example* function?
  - a)  $c == 12$
  - b)  $c-3 > 12 \wedge d < c*2$**
  - c)  $m == 12$
  - d)  $answer == 3*c + 2*(d-1)$
  
9. What must we prove just before the end of the *example* function?
  - a)  $answer == example(m, m+n)$
  - b)  $Res[Z] == answer$
  - c)  $answer == 3*c + 2*(d-1)$**
  - d)  $c-3 > 12 \wedge d < c*2$
  
10. What can we claim as a premise just after we call *example* in the test code?
  - a)  $ans == 3*m + 2*(m+n-1)$**
  - b)  $ans == 3*c + 2*(d-1)$
  - c)  $ans == example(m, m+n)$
  - d)  $answer == 3*c + 2*(d-1)$

**Open-Ended. 4 questions, 16-18 points each.**

1. (16 pts) Consider the following function. Assume that the loop invariants have already been shown to still hold at the end of each iteration – you do not need to add any deduce blocks inside the loop. Complete deduce blocks **PART 1** and **PART 2** so that function would be verified as correct in Logika.

```
def addAll(a: Z, b: Z): Z = {
  Contract(
    Requires( b ≥ 0 ),
    Ensures( Res[Z] == a - 4*b - 4 )
  )
  var amount: Z = a - 4
  var counter: Z = 1
  Deduce( [PART 1]
    1 ( amount == a - 4 ) by Premise,
    2 ( counter == 1 ) by Premise,
    3 ( amount == a - 4*counter ) by Algebra*(1,2)
    4 ( b ≥ 0 ) by Premise,
    5 ( counter ≤ b + 1 ) by Algebra*(2,4)
  )
  while (counter ≤ b) {
    Invariant(
      Modifies (amount, counter),
      amount == a - 4*counter,
      counter ≤ b + 1
    )
    amount = amount - 4
    counter = counter + 1
    //assume proof blocks inside the loop are complete
  }
  Deduce( [PART 2]
    1 ( amount == a - 4*counter ) by Premise,
    2 ( counter ≤ b + 1 ) by Premise,
    3 ( ¬( counter ≤ b ) ) by Premise,
    4 ( counter > b ) by Algebra*(3),
    5 ( counter == b + 1 ) by Algebra*(2,4)
    6 ( amount == a - 4*b - 4 ) by Algebra*(1,5)
  )
  return amount
}
```

2. (18 pts) Use mathematical induction to prove that for every natural number  $n$ :

$$5 + 10 + \dots + 5n = \frac{5n(n+1)}{2}$$

**Complete the base case and inductive step below.**

**Base case:**

We must prove the claim for  $n = 1$ , which is the smallest natural number. We have that:

$$5 \cdot 1 = 5$$

$$\text{And that: } \frac{5n(n+1)}{2} = \frac{5 \cdot 1 \cdot 2}{2} = 5.$$

The two sides are equal, so the base case holds.

**Inductive step.** We assume the inductive hypothesis, that (write the inductive hypothesis in the space below):  $5 + 10 + \dots + 5k = \frac{5k(k+1)}{2}$  for some fixed natural number  $k$ .

(Now complete the inductive step). We must show that our claim holds for  $k+1$  – that is, that:  $5 + 10 + \dots + 5k + 5(k+1) = \frac{5(k+1)((k+1)+1)}{2}$ . We have that:

$$\begin{aligned} 5 + 10 + \dots + 5k + 5(k+1) &= \frac{5k(k+1)}{2} + 5(k+1) \\ &= \frac{5k(k+1) + 10(k+1)}{2} \\ &= \frac{5(k+1)(k+2)}{2} \\ &= \frac{5k(k+1)((k+1)+1)}{2} \end{aligned}$$

The inductive step holds.

Thus we have shown our claim holds for all natural numbers.

3. (18 pts) Consider the following questions about recursive definitions.

a) (4 pts) Consider the following recursive definition of  $g(n)$ , where  $n \in \mathbb{Z}$  and  $n \geq -1$ :

$$g(-1) = 5$$

$$g(n) = g(n-1) - n, n > -1$$

**Find  $g(2)$ .** Show your work and put a box around your final answer.

$$g(0) = g(-1) - 0 = 5 - 0 = 5$$

$$g(1) = g(0) - 1 = 5 - 1 = 4$$

$$g(2) = g(1) - 2 = 4 - 2 = 2$$

$$\mathbf{g(2) = 2}$$

b) (4 pts) Consider the pattern 3, 10, 17, 24,... Give a recursive definition of the function  $f(n)$ , where  $n \in \mathbb{N}$ , that defines the pattern.

$$\mathbf{f(1) = 3}$$

$$\mathbf{f(n) = f(n-1) + 7, n > 1}$$

c) (5 pts) Give a recursive definition of the function  $h(n)$ , where  $h(n)$  is the sum of the first  $n$  positive even integers. (Note: 0 is not considered to be positive.)

$$\mathbf{h(1) = 2}$$

$$\mathbf{h(n) = h(n-1) + 2n, n > 0}$$

d) (5 pts) Give a recursive definition of the set  $S$ , which includes all odd numbers between 50 and 100. (For example,  $57 \in S$  but  $11 \notin S$ .)

$$\mathbf{51 \in S}$$

$$\mathbf{x \in S \wedge x+2 \leq 100 \rightarrow x+2 \in S}$$

4. (18 pts) Consider the following function. Complete the deduce blocks below to verify that the function satisfies its postcondition.

```
def example(a: Z): Z = {
  Contract(
    Requires( a ≥ 10 ),
    Ensures ( Res[Z] ≥ 25 )
  )
  var total: Z = a
  if (a < 16) {
    total = total * 2
    Deduce(
      1 ( a < 16 ) by Premise,
      2 ( a ≥ 10 ) by Premise,
      3 ( Old(total) == a ) by Premise,
      4 ( total == Old(total) * 2 ) by Premise,
      5 ( total ≥ 20 ) by Algebra*(2, 3, 4)
    )
  } else {
    total = total + 4
    Deduce(
      1 ( ¬( a < 16 ) ) by Premise,
      2 ( Old(total) == a ) by Premise,
      3 ( total == Old(total) + 4 ) by Premise,
      4 ( total ≥ 20 ) by Algebra*(1,2,3)
    )
  }
  Deduce(
    1 ( total ≥ 20 ) by Premise
  )
  total = total + 5
  Deduce(
    1 ( Old(total) ≥ 20 ) by Premise,
    2 ( total == Old(total) + 5 ) by Premise,
    3 ( total ≥ 25 ) by Algebra*(1,2)
  )
  return total
}
```